

# Playwright Self Learning

Playwright Project Create:

Create a folder for project. Open terminal in that folder. Type this command:

```
npm init playwright@latest
```

Essential Command:

```
npx playwright test
```

Runs the end-to-end tests.

```
npx playwright test --ui
```

Starts the interactive UI mode.

```
npx playwright test --project=chromium
```

Runs the tests only on Desktop Chrome.

```
npx playwright test example
```

Runs the tests in a specific file.

```
npx playwright test --debug
```

Runs the tests in debug mode.

```
npx playwright codegen
```

Auto generate tests with Codegen.

We suggest that you begin by typing:

```
npx playwright test
```

For my test running command

```
npx playwright test product.spec.js --headed --project chromium --workers=1
```

For report show:

```
npx playwright show-report
```

```
command: npx playwright codegen // For record & generating code by browsing t
*
```

Test file:

```
import {test,expect} from '@playwright/test';

test("Locator Test Check", async({page}) =>{

  await page.goto("AB.com");
  await page.locator("#locator").click();
  await page.locator("#UsernameLocator").fill("DJ");

})
```

## Checkboxes and radio buttons

Using `locator.setChecked()` is the easiest way to check and uncheck a checkbox or a radio button. This method can be used

```
// Check the checkbox
await page.locator('#Checkbox').check();
// Assert the checked state
await expect(await page.locator('#Checkbox')).toBeChecked();
```

Assertion:

```
1  Assertions
2  -----
3  Playwright includes test assertions in the form of expect function.
4
5
6  https://playwright.dev/docs/test-assertions
7
8
9  1) expect(page).toHaveURL()           Page has URL
10
11 2) expect(page).toHaveTitle()         Page has title
12
13 3) expect(locator).toBeVisible()      Element is visible
14
15 4) expect(locator).toBeEnabled()      Control is enabled
16    expect(locator).toBeDisabled()    Element is disabled
17
18 5) expect(locator).toBeChecked()      Radio/Checkbox is checked
19
20 6) expect(locator).toHaveAttribute()  Element has attribute
21
22 7) expect(locator).toHaveText()       Element matches text
23
24 8) expect(locator).toContainText()    Element contains text
25
26 9) expect(locator).toHaveValue(value) Input has a value
27
28 10) expect(locator).toHaveCount()     List of elements has given length
```

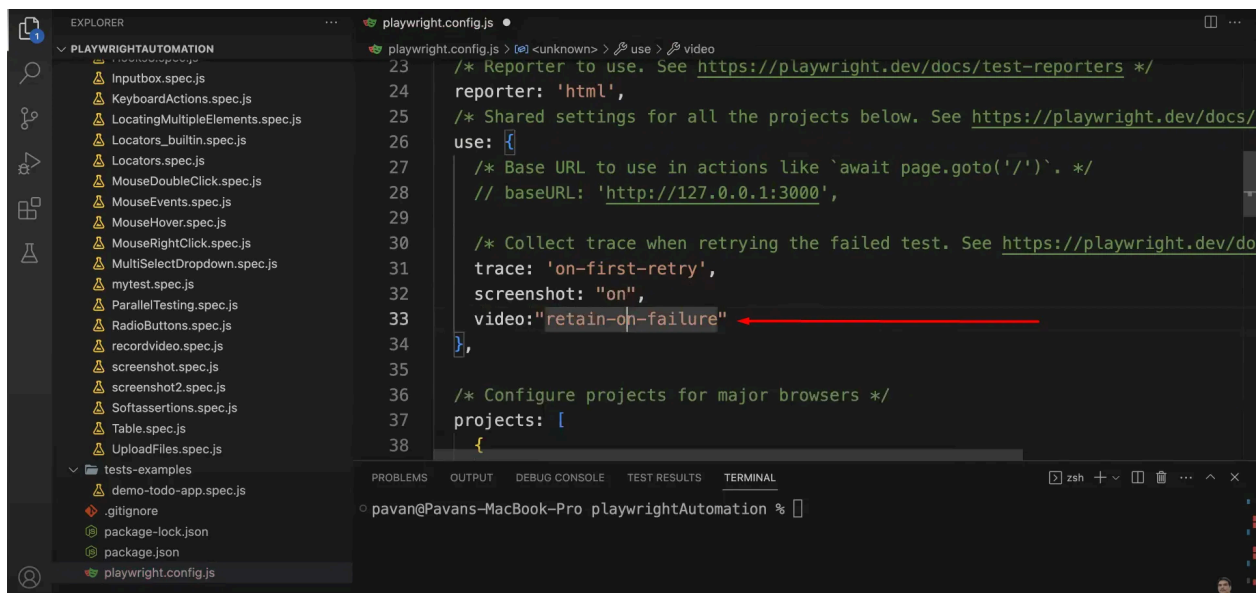
For take screenshot of failed test case for reporter:

Add the screenshot property in the playwright.config.js

```
/* Shared settings for all the projects below. See https://playwright.dev/docs/
use: {
  /* Base URL to use in actions like `await page.goto('/')`. */
  // baseURL: 'http://127.0.0.1:3000',

  /* Collect trace when retrying the failed test. See https://playwright.dev/docs/trace-viewer
  trace: 'on-first-retry',
  screenshot: 'on'
},
```

For taking video add video property in the similar file:



Element CSS Selector:

if dropdown or search option is working in Realtime then delete these values in the event listeners:

**\*\*\*For those who cannot find 'blur' event, alternatively you can remove ones in 'focusout'**

```
//button/i[contains(text(), "ABC")]
```

Another way to get text:

```
await $('button = Add Text')
```

Right button click:

```
await variable.click({ button: 'right' })
```

`await browser.keys(["Control","a"]);` — it means it will select all the text. when performing multiple keyboard action then it is written like this.

Tab switching:

```
test("Verify clicking the footer Twitter button redirects to Twitter and returns.", as
const productPage = new ProductPage(page);
const [newPage] = await Promise.all([
  context.waitForEvent('page'), // Wait for a new tab to open
  productPage.productpagefootertwitterBtnAutomate()
]);

await newPage.waitForLoadState();
```

```

await expect(newPage).toHaveTitle("Sauce Labs (@saucelabs) / X");
await newPage.close(); // Close new tab
await page.bringToFront(); // Bring original page back
});

```

Handle alert:

`await browser.acceptAlert();` //for accept the alert.

`await browser.dismissAlert();` //for not accepting or dismiss the alert

`await browser.sendAlertText("WebdriverIO is fun");` // for sending the text in input box of alert.

`isAlertOpen()` //return the boolean value

`getAlertText()` // returns the alert text

XPath:

## Xpath Methods

1. We can catch everything that we caught using CSS selector (id, class, attribute, text)
2. Contains() method
  - a. `//button[contains(@type,'sub')]`
3. And / Or in xpath
  - a. `//*[@type='submit' or @name='btnReset']`
4. Starts with
  - a. `//a[starts-with(@href,'http://www.cefalo.com')]`
  - b. `Logbt andvjhad`
  - c. `//button[starts-with(@id,'logbt')]`
5. Text() method
  - a. `//td[text()='Delete']`
6. Following
  - a. `//button[@type='sub']/following::input`
7. Child
  - a. `//form[@id='java_technologies']/child::li`
8. Following-sibling
  - a. `//div[@type='submit']/following-sibling::input`
9. Parent
  - a. `//div[@id='someid']/parent::div`

dragelementvariable.dragAndDrop(dropementVariable); ///for drag and drop an element but these value stored in function...so need to store in a variable.

```
async dragTheButtonAndDropInsideTheDiv() {  
  const dragElement = await this.theButtonIwantToDrag  
  const dropTarget = await this.thePlaceIwantToDrop  
  dragElement.scrollIntoView()  
  await browser.pause(3000)  
  dragElement.dragAndDrop(dropTarget)  
}
```

elementvariable.moveTo() //for mouse hover

## POM Structure:

Element & Action

pages Folder:

1st file: productPage.js

```
exports.ProductPage = class ProductPage{  
  
  constructor(page) {  
    this.page = page;  
    this.labbackpackElement= page.locator("#add-to-cart-sauce-labs-backpac  
    this.bikelightElement = page.locator("#add-to-cart-sauce-labs-bike-light");  
    this.cartElement = page.locator("a[class='shopping_cart_link']");  
    this.filterbtn = page.locator('select[class="product_sort_container"]');  
    this.filterverification = page.locator("//button[@id='add-to-cart-sauce-labs-  
    this.hamburgerbtn = page.locator("//button[@id='react-burger-menu-btn']"  
    this.hamburgerBtnVerification = page.locator("//a[@id='reset_sidebar_link']"  
    this.twitterBtn = page.locator('a[data-test="social-twitter"]');  
    this.twitterBtn = page.locator('a[data-test="social-twitter"]');  
    this.fbBtn = page.locator('a[data-test="social-facebook"]');  
    this.LinkedinBtn = page.locator('li[class="social_linkedin"]');
```

```

    }

//Action on Elements
async productpagefootertwitterBtnAutomate(){
    await this.twitterBtn.click();
}

async productpagefooterfacebookBtnAutomate(){
    await this.fbBtn.click();
}

async productpagefooterlinkedinBtnAutomate(){
    await this.LinkedinBtn.click();
}

async productpagefilter(){
    await this.filterbtn.click();
    await this.filterbtn.selectOption("Price (low to high)");
}

async actionHamburger(){
    await this.hamburgerbtn.click();
}

async productpageAutomate(){
    await this.labbackpackElement.click();
    await this.bikelightElement.click();
    await this.cartElement.click();
}

```



```
}
```

pages Folder:

2nd file: cartPage.js

```
exports.CartPage = class CartPage{

  constructor(page) {
    this.page = page;
    this.removeLabBackpackElement = page.locator("#remove-sauce-labs-backpack");
    this.checkoutElement = page.locator("#checkout");
  }

  //Action on Elements

  async cartpageAutomate(){
    await this.removeLabBackpackElement.click();
    await this.checkoutElement.click();
  }

}
```

Now creating the test file:

test folder: (Create only a single file where all test remained)

product.spec.js

```
import { test, expect } from '@playwright/test';
const { ProductPage } = require("../pages/productPage.js");
const { CartPage } = require("../pages/cartPage.js");
```

```
let page, context; // Global page and context instance
```

```
test.describe('E2E Test Suite', () => {
  test.beforeAll(async ({ browser }) => {
    context = await browser.newContext(); // Creates a single browser context f
    page = await context.newPage(); // Single page instance for all tests
    await page.goto("https://www.saucedemo.com/");
    await page.locator('input[id="user-name"]').fill("standard_user");
    await page.locator('input[id="password"]').fill("secret_sauce");
    await page.locator("#login-button").click();
  });

  test.afterAll(async () => {
    await page.close(); // Close the page after all tests
    await context.close(); // Close the browser context
  });

  test.describe('Product Page Test', () => {
    test("Verify it is visited in correct URL.", async () => {
      await expect(page).toHaveURL("https://www.saucedemo.com/inventory.l
    });

    test("Verify clicking the footer Twitter button redirects to Twitter and returns
      const productPage = new ProductPage(page);
      const [newPage] = await Promise.all([
        context.waitForEvent('page'), // Wait for a new tab to open
        productPage.productpagefootertwitterBtnAutomate()
      ]);

      await newPage.waitForLoadState();
      await expect(newPage).toHaveTitle("Sauce Labs (@saucelabs) / X");
      await newPage.close(); // Close new tab
      await page.bringToFront(); // Bring original page back
    });
  });
});
```

```

test("Verify clicking the footer Facebook button redirects to Facebook and r
  const productPage = new ProductPage(page);
  const [newPage] = await Promise.all([
    context.waitForEvent('page'),
    productPage.productpagefooterfacebookBtnAutomate()
  ]);

  await newPage.waitForLoadState();
  await expect(newPage).toHaveTitle("Sauce Labs | Facebook");
  await newPage.close(); // Close new tab
  await page.bringToFront();
});

test("Verify clicking the footer LinkedIn button redirects to LinkedIn and retu
  const productPage = new ProductPage(page);
  const [newPage] = await Promise.all([
    context.waitForEvent('page'),
    productPage.productpagefooterlinkedinBtnAutomate()
  ]);

  await newPage.waitForLoadState();
  await expect(newPage).toHaveURL("https://www.linkedin.com/company/");
  await newPage.close(); // Close new tab
  await page.bringToFront();
});

test("Verify Cart Page Sorting is working low to high", async () => {
  const productPage = new ProductPage(page);
  await productPage.productpagefilter();
  await expect(productPage.filterverification).toBeVisible();
});

test("Verify Hamburger Button is Working", async () => {
  const productPage = new ProductPage(page);
  await productPage.actionHamburger();
  await expect(productPage.hamburgerBtnVerification).toBeVisible();
});

```

```

});

test("Verify Add Product to cart", async () => {
  const productPage = new ProductPage(page);
  await productPage.productpageAutomate();
});

});

test.describe('Cart Page Test', () => {
  test.beforeEach(async () => {
    await page.goto("https://www.saucedemo.com/cart.html"); // Navigate to cart page
  });

  test("Verify it is visited in correct URL.", async () => {
    await expect(page).toHaveURL("https://www.saucedemo.com/cart.html");
  });

  test("Verify Remove a product and checkout the page", async () => {
    const cartPage = new CartPage(page);
    await cartPage.cartpageAutomate();
  });
});
});

```

```

async dragTheButtonAndDropInsideTheDiv() {
  const dragElement = await this.theButtonIWantToDrag
  const dropTarget = await this.thePlaceIWantToDrop
  dragElement.scrollIntoView()
  await browser.pause(3000)
  dragElement.dragAndDrop(dropTarget)
}

```

## Assertion:

### Browser assertion

1. toHaveUrl()
2. toHaveTitle()

```
await expect(browser).toHaveUrl("https://play1.automationcamp.ir/order_submit.html")
//await expect(browser).toHaveTitle('Loginff')
```

### Element matcher

toBeDisplayed (Exist + Present)

isExist() //maybe it is present in dom, but showed later after an action.

toBePresent

toBeClickable

toBeFocused

toHaveAttribute('attname','attvalue')

expect(input).toHaveValue("Nameapnar")

toBeDisabled

toBeEnabled

toBeSelected

toBeChecked

toHaveText

toBeDisplayedInViewport

Error message

### HOME WORK

toBeElementsArrayOfSize

toHaveSize

```
const triggerButton = await $('#visibility_trigger')
const clickMeButton = await $('#visibility_target')
await expect(clickMeButton).toBeDisplayed()
```

We can add custom message for failed case in assertion.

```
await expect(disableButton).toBeDisplayedInViewport({message : 'I did not scroll to that element that is why the button din not get in the view'})
```

### String Manipulation at the assertion:

1. String option
  - a. ignoreCase
  - b. trim
  - c. Containing
  - d. atStart
  - e. atEnd
2. Number option
  - a. eq
  - b. lte
  - c. Gte

**ignoreCase:** It will ignore the text as case sensitive.

```
it("Practicing String Option for Element Matcher Assertion", async () => {
  await browser.url('https://letcode.in/test')
  await browser.pause(5000)
  const firstHeaderElement = await $('//header/p[1]')
  await expect(firstHeaderElement).toHaveText('In',{ignoreCase:true})
})
```

trim: It will trim the whitespace after or before the string.

```
it("Practicing String Option for Element Matcher Assertion", async () => {
  await browser.url('https://letcode.in/test')
  await browser.pause(5000)
  const firstHeaderElement = await $('//header/p[1]')
  await expect(firstHeaderElement).toHaveText('In',{ignoreCase:true, trim:true})
})
```

## Class-10-Mock,Shadow Element, Browser Event

Browser Events:

```
// Browser Events =====
it.only('Browser Events', async () => {
  await browser.navigateTo('https://play1.automationcamp.ir/')
  await browser.pause(3000)
  await browser.url('https://letcode.in/test');
```

```

    await browser.pause(3000)
    await browser.back()
    await browser.pause(3000)
    await browser.forward()
    await browser.pause(3000)
    await browser.refresh()
    await browser.pause(3000)
  })

```

Shadow Element:

```

// Shadow Elements =====
it('Shadow Elements', async () => {
  await browser.url('https://practice.expandtesting.com/shadowdom');
  await browser.pause(3000)
  const innerEl = await $('#shadow-host').shadow$('#my-btn');
  await innerEl.click()
  await browser.pause(3000)
})

it('Shadow Elements', async () => {
  await browser.url('http://uitestingplayground.com/shadowdom');
  await browser.pause(3000)
  const innerEl = await $('#guid-generator').shadow$('#editField');
  await innerEl.click()
  await innerEl.setValue("zjcfjkasnkjfnalksjnl")
  await browser.pause(3000)
})

```

Mock:

```

describe('Mock Request Example and other', () => {
  it('should mock a request and send us joke', async () => {

```



```

await browser.url('https://jokeapi.dev/');

const mock = await browser.mock('https://v2.jokeapi.dev/joke/Any', {method: 'GET'})

mock.respond({
  "error": false,
  "category": "Programming",
  "type": "single",
  "joke": "dfsdfsdfsdf",
  "flags": {
    "nsfw": false,
    "religious": false,
    "political": false,
    "racist": false,
    "sexist": false,
    "explicit": false
  },
  "id": 51,
  "safe": true,
  "lang": "en"
}, {
  statusCode: 200,
  headers: { 'Content-Type': 'application/json' }
});

const sendRequestButton = await $('//button[contains(text()),"Send Request"]');
await sendRequestButton.scrollIntoView()
await browser.pause(3000)
await sendRequestButton.click()
await browser.pause(10000)

```

```

});
    //const someVariable = browser.mock(url, { method, headers, responseHeader, responseText })
    // somevariable now has the mock
    // then I have to say him after mocking what data/step he needs to do.

    it('should mock a request to JSONPlaceholder', async () => {

        await browser.url('https://jsonplaceholder.typicode.com/');

        const mockRunButton = await browser.mock('https://jsonplaceholder.typicode.com/posts/1', {
            method: 'GET',
            response: {
                status: 200,
                body: {
                    "userId": 1,
                    "id": 1,
                    "title": "Shorkari chakri korile desher sheba kora jay. Onno job behuda",
                    "completed": false
                }
            }
        }, {statusCode: 200, headers: { 'Content-Type': 'application/json' } });

        const runButton = await $('#run-button')
        await runButton.scrollIntoView()
        await browser.pause(3000)
        await runButton.click()
        await browser.pause(10000)

    });

```

```

it('should mock a request and send us joke', async () => {
  await browser.url('https://play1.automationcamp.ir/register.html');

  const mock = await browser.mock('https://play1.automationcamp.ir/confirm',
    method: 'get'
  });

  const htmlResponse = `
    <html>
      <body>
        <h1>Mocked HTML Content</h1>
        <p>This is a mocked HTML response.</p>
      </body>
    </html>
  `;

  mock.respond(htmlResponse, {
    statusCode: 200,
    headers: { 'Content-Type': 'text/html' }
  });

  const firstName = await $('input[name="first_name"]');
  const lastName = await $('input[name="last_name"]');
  const email = await $('input[name="email"]');
  const password = await $('input[name="password"]');
  const confirmPassword = await $('input[name="confirm_password"]');
  const checkBox = await $('input[name="terms"]');
  const registerBtn = await $('#submit_button')

  await firstName.setValue("Noman")
  await lastName.setValue("Ibrahim")
  await email.setValue("Ibrahim@mail.com")
  await password.setValue("123456")
  await confirmPassword.setValue("123456")

```

```
await checkBox.click()
await registerBtn.click()

await browser.pause(10000)
});
```

## Class 11: Cache\_Cookie\_Debugger\_Array

## Class 12: Flash Message, Flash Drop-down

wait for:

If I do not know how much time to take a element to show then we can use these instead for using **browser.wait()**. **Because browser.wait() always makes the exact waiting.... If the elements found in that waiting time then it is also completes it waiting time but lower timeout method will proceed the next step as soon as it found the element.**

```
elem.waitForDisplayed({ timeout: 3000 })
elem.waitForClickable({ timeout: 3000 })
elem.waitForExist({ timeout: 3000 })
elem.waitForEnabled({ timeout: 3000 })
//https://play1.automationcamp.ir/expected_conditions.html
```

## Class 13: Mocha, Calender

Calendar:

<https://primeng.org/calendar>

<https://www.globalsqa.com/demo-site/datepicker/#Icon%20Trigger>

//div[@class="ant-picker-calendar-date-value"][text()='14"]

<https://ant.design/components/calendar>

```
document.getElementById('inputField').value = "ksndlskndlsk"  
span[@data-date="2024-6-4"]
```

```
it.only('Calendar ', async () => {  
  await browser.url('http://127.0.0.1:5500/claendar.html');  
  await browser.pause(3000)  
  const calendarInput = await $('#date-input')  
  await calendarInput.setValue("1/7/2024")  
  await browser.pause(3000)  
})
```

```
it('Calendar ', async () => {  
  await browser.url('http://127.0.0.1:5500/claendar.html');  
  await browser.pause(3000)  
  const calendarInput = await $('#date-input')  
  await browser.execute((input) => {input.value = '12/07/2023';}, calendarInput);  
  await browser.pause(3000)  
})
```

```
it.only('Calendar ', async () => {  
  await browser.url('https://primeng.org/calendar');  
  await browser.pause(1000)  
  const calendarInput = await $$("//span/input[@type='text']")  
  await calendarInput[0].scrollIntoView()  
  // await calendarInput[0].click()
```

```

const isClickable = await calendarInput[0].isClickable()
console.log("===== ",isClickable)
await calendarInput[0].click()
await browser.pause(1000)
await browser.execute((input) => {
  input.value = '01/2024';
}, calendarInput[0]);
await browser.pause(2000)
await browser.keys([Key.Enter])
await browser.pause(2000)

})

it.only('Calendar ', async () => {
  await browser.url('https://primeng.org/calendar');
  await browser.pause(1000)
  const calendarInput = await $$("//span/input[@type='text']")
  await calendarInput[13].scrollIntoView()
  // await calendarInput[0].click()
  const isClickable = await calendarInput[13].isClickable()
  console.log("===== ",isClickable)
  await calendarInput[13].click()
  await browser.pause(1000)
  await browser.execute((input) => {
    input.value = '01/2024';
  }, calendarInput[13]);
  await browser.pause(2000)
  await browser.keys([Key.Enter])
  await browser.pause(2000)

})

it.only('Calendar ', async () => {
  await browser.url('https://primeng.org/calendar');
  await browser.pause(1000)
  const calendarInput = await $$("//span/input[@type='text']")
  await calendarInput[17].scrollIntoView()

```

```

// await calendarInput[0].click()
const isClickable = await calendarInput[17].isClickable()
console.log("===== ",isClickable)
// await calendarInput[17].click()
await browser.pause(1000)
await browser.execute((input) => {
  input.value = '07/10/2024';
}, calendarInput[17]);
await browser.pause(2000)
await browser.keys([Key.Enter])
await browser.pause(2000)

})

it.only('Calendar ', async () => {
  await browser.url('https://www.globalsqa.com/demo-site/datepicker/');
  await browser.pause(1000)
  const iframe = await $('//iframe[contains(@data-src,'datepicker/default')]')
  await browser.switchToFrame(iframe)
  await browser.pause(1000)
  const dateFiled = await $('#datepicker')
  await browser.execute((input) => {
    input.value = '07/10/2024';
  }, dateFiled);
  await browser.pause(2000)
  // await browser.keys([Key.Enter])
  await browser.pause(2000)

})

it.only('Calendar', async () => {
  await browser.url('https://www.globalsqa.com/demo-site/datepicker/#DropDo
  await browser.pause(5000)
  const iframe = await $('//iframe[contains(@data-src,'datepicker/dropdown')]')
  await browser.switchToFrame(iframe)
  await browser.pause(1000)

```

```

const dateFiled = await $("#datepicker")
await browser.execute((input) => {
  input.value = '07/10/2024';
}, dateFiled);
await browser.pause(2000)
// await browser.keys([Key.Enter])
await browser.pause(2000)
})

```

nth child chaining

```

it.only('Table', async function () {
  await browser.url('http://127.0.0.1:5500/table.html');
  await browser.pause(2000)
  const firstRow = await browser.$('#employeeTable tbody tr:nth-child(1)');
  const id = await firstRow.$('td:nth-child(1)').getText();
  console.log("===== ",id)
})

```